

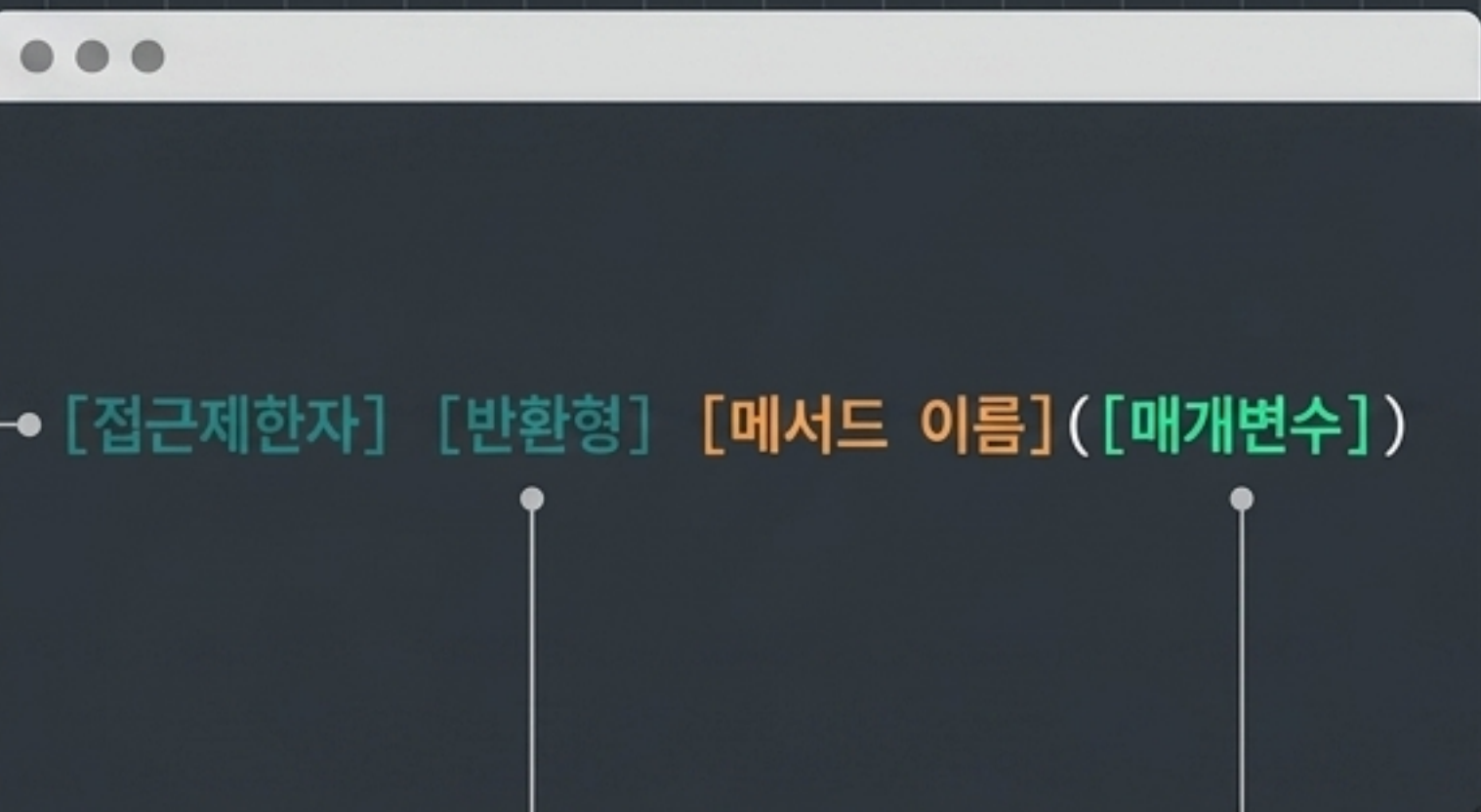
C# 클래스의 해부학

Anatomy of a C# Class

메서드, 생성자, 그리고 객체 지향의 구조적 설계도
A Blueprint for Methods, Constructors, and Object-Oriented Structure

- 접근제한자 (Access Modifier):
기계를 만질 수 있는 권한

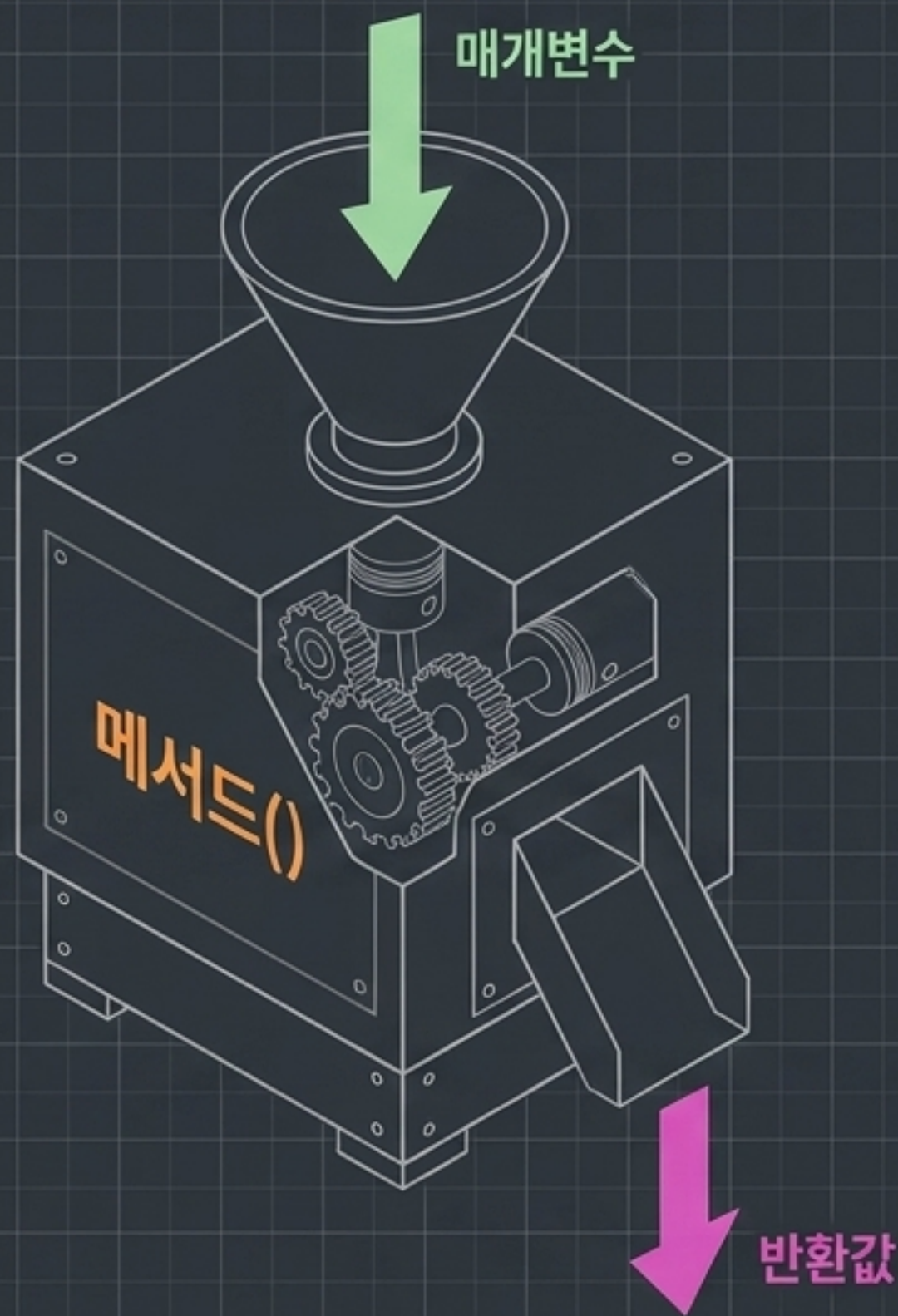
반환형 (Return Type):
기계가 배출하는 결과물의 종류



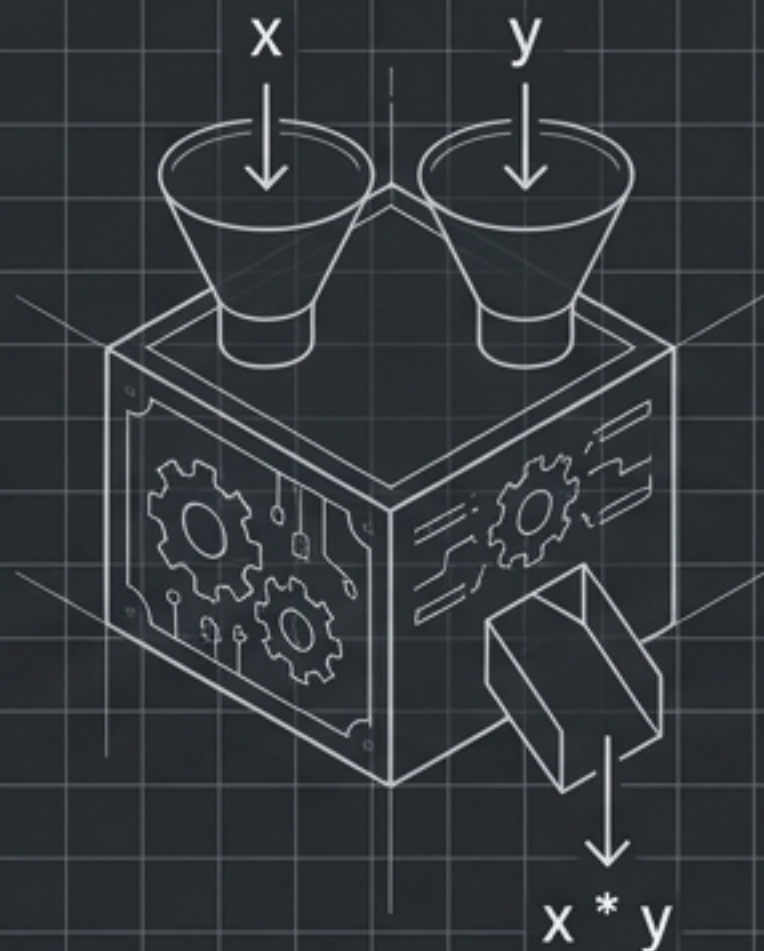
• [접근제한자] [반환형] [메서드 이름] ([매개변수])

- 반환형 (Return Type):
기계가 배출하는
결과물의 종류

- 매개변수 (Parameters):
기계를 작동시키기 위해
투입하는 재료

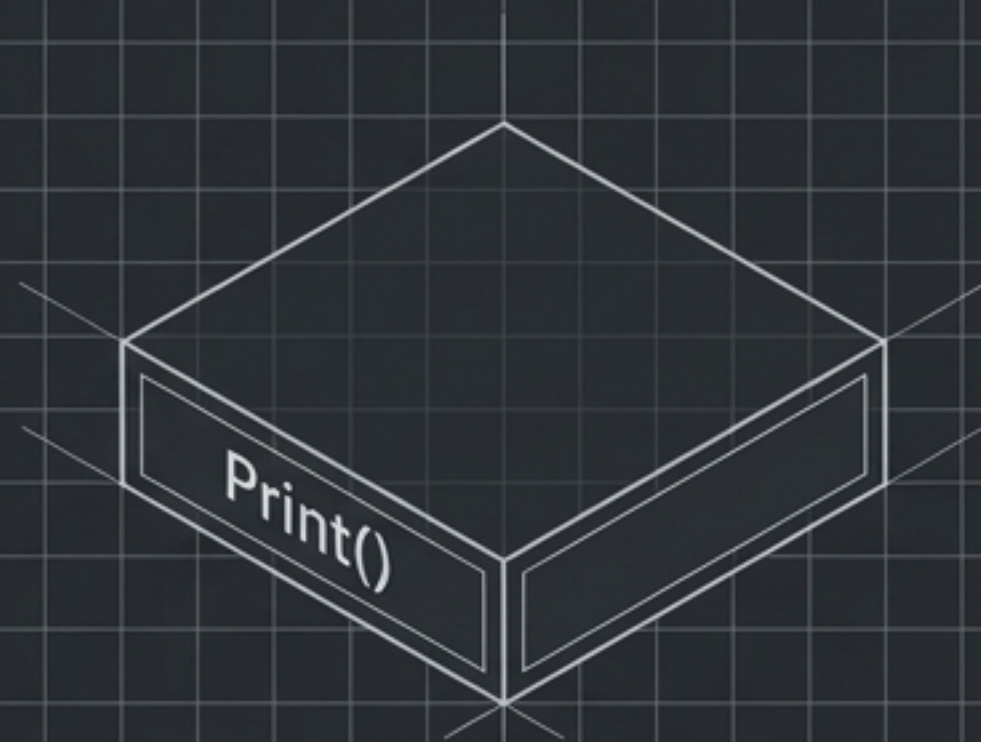


입력과 배출의 변형 (Variations in Input & Output)



```
public int Multi(int x, int y)
```

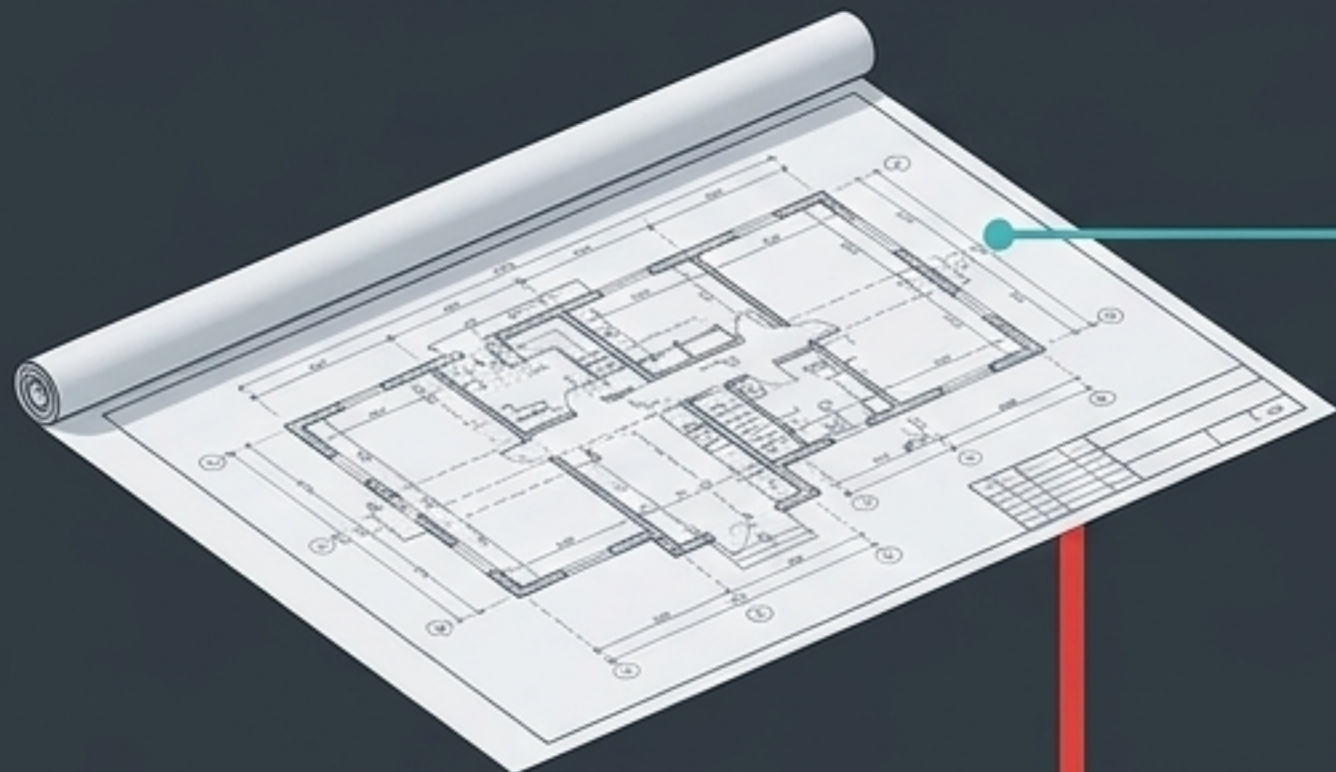
다중 매개변수: 콤마(,)를 사용해 여러 개의 재료를 투입 가능.



```
public void Print()
```

void (빈 공간): 아무것도 반환하지 않는 메서드. 계산 결과를 배출하지 않고, 내부 동작(예: 화면 출력)만 수행.

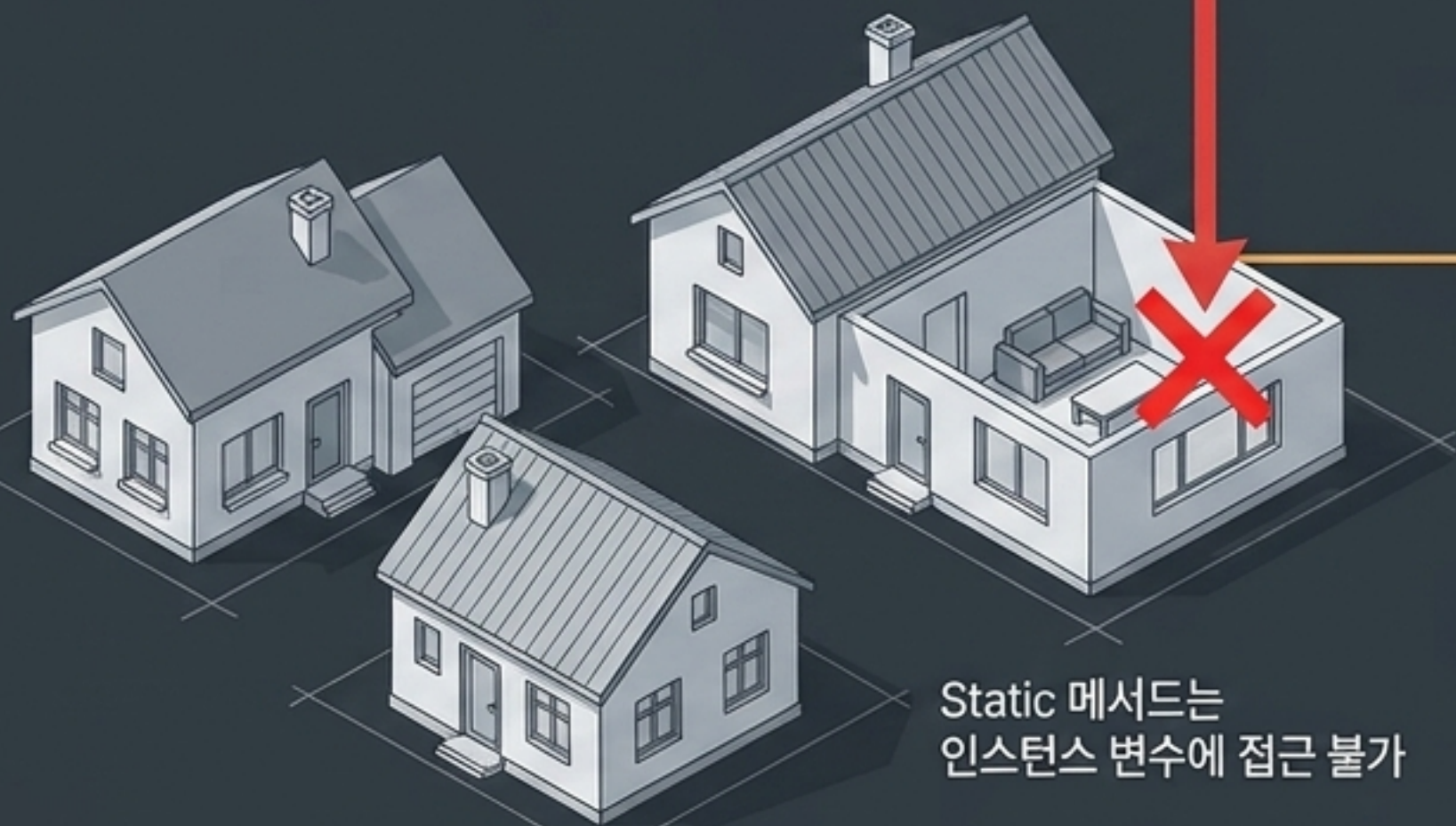
인스턴스 vs. 클래스(Static) (Scaling Production)



static (클래스 레벨)

클래스(Static) 메서드: **static** 키워드 사용.
설계도 자체에 내장된 공용 기능.
집을 짓기 전(프로그램 실행 순간)에 이미 메모리에 존재.
Math.Abs()가 대표적 예시.

```
static void Method(inputs, outputs)
```



new (인스턴스 레벨)

인스턴스 메서드: 집을 지은 후(new),
그 집 안에서만 사용하는 전용 기능.

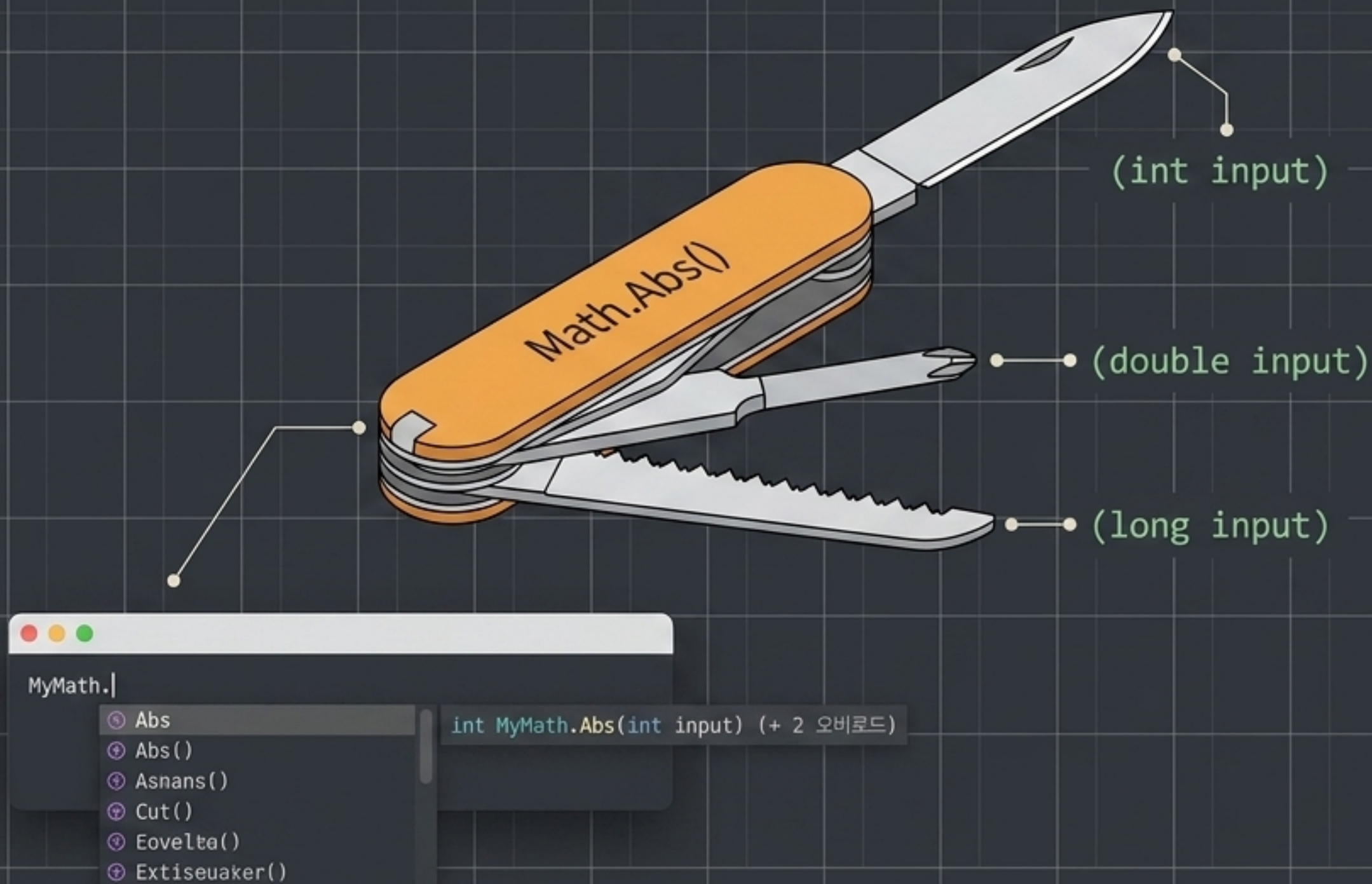
```
static void MethodAbsseeies((new), inputs, outputs)
```

Static 메서드는
인스턴스 변수에 접근 불가

핵심 진단 매트릭스: 인스턴스 vs 클래스 메서드

구분 (Dimension)	인스턴스 (Instance)	클래스 (Static)
호출 문법	test.Method() (객체 이름 사용)	Class.Method() (클래스 이름 직접 사용)
메모리 할당 시점	new 키워드로 객체를 생성할 때	프로그램이 실행되는 순간 (자동)
인스턴스 변수 접근	가능 (Yes)	절대 불가 (No) - 아직 존재하지 않는 객체의 데이터에 접근할 수 없음
Main() 메서드	N/A	C#의 진입점 Main()은 항상 static <pre>static void Main(string[] args) { }</pre>

유연성의 확보: 오버로딩 (Method Overloading)



오버로딩(Overloading) :
이름은 같지만, 매개변수의
형태(자료형)나 개수가 다른
메서드를 여러 개 생성하는
기법.

입력되는 데이터에 따라
컴파일러가 알맞은 '도구'를
자동으로 선택하여 동작.

오버로딩의 한계 (The Limits of Overloading)

⚠ 오류 발생 (CS0111)

```
public int Test(int input) {}
```



```
public double Test(int input) {}
```

메서드의 이름과 매개변수가 완전히 동일하다면, 반환값(Return Type)만 다르게 설정하는 것은 불가능합니다.
이유: 컴파일러는 메서드를 호출할 때 전달되는 '매개변수'를 보고 어떤 메서드를 실행할지 결정하기 때문입니다.

접근 제어: Public vs Private (Access Modifiers)



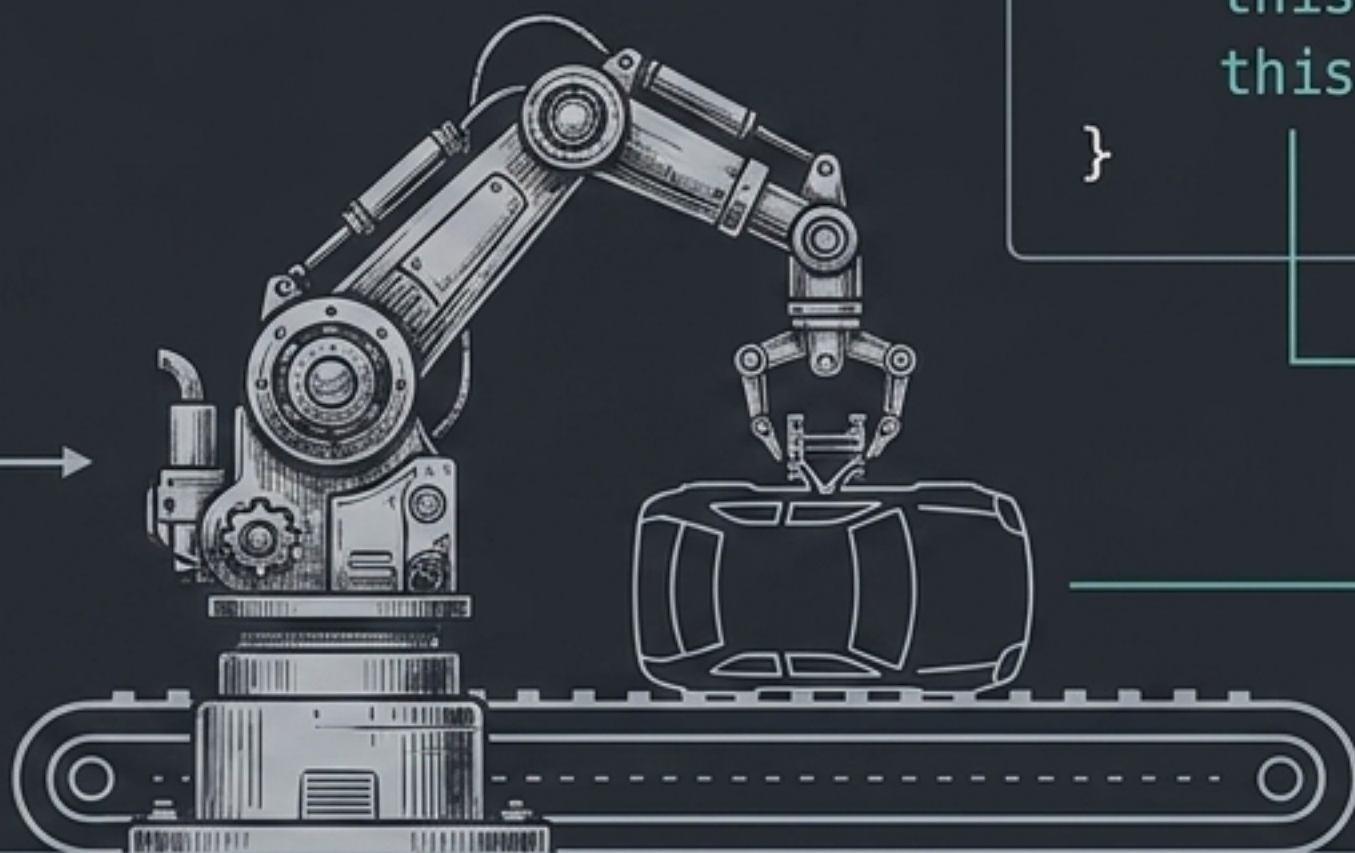
private (비공개): 기본값(생략 시 자동 적용).
자신의 클래스 내부에서만 접근 가능한 자물쇠 채워진 상태.



public (공개): 외부의 다른 클래스에서도
자유롭게 호출하고 접근할 수 있는 개방 상태.

조립 라인: 인스턴스 생성자 (The Constructor)

new



```
public Product(string name, int price)
{
    this.name = name;
    this.price = price;
}
```

this 키워드는 클래스 자기 자신을 나타냅니다.
따라서 this.name은 자신의 name입니다.

this.price는 자신의 price입니다.

인스턴스를 생성(new)할 때
가장 먼저, 자동으로 한 번
호출되는 특수 메서드.

3대 조건:

1. 클래스와 이름이 완벽히 동일
2. 접근 제한자는 보통 public
3. 반환형(void 포함)을 아예 적지 않음.

this 키워드:

클래스 자기 자신을 가리키며,
매개변수와 내부 변수(필드)의
이름이 같을 때 구분하는 역할.

특수 생성자 (Advanced Constructors)

```
class Hidden
{
    private Hidden() { }
}
```



```
Hidden hidden = new Hidden();
```

외부에서 생성 원천 차단

Private 생성자: 외부에서 new로 인스턴스를 아예 만들 수 없게 원천 차단할 때 사용 (정적 멤버 전용 클래스나 팩토리 패턴 활용).

```
static Sample()
{
    value = 10;
}
```

```
Console.WriteLine(Sample.value);
```

이때 단 한 번만 호출됩니다

정적(Static) 생성자: 접근 제한자(public 등)나 매개변수를 가질 수 없음.
프로그램 실행 후 해당 클래스를 최초로 사용하는 단 한 번의 순간에만 자동 호출됨.

해체와 정리: 소멸자 (The Destructor)



Garbage Collector

```
class Product
{
    ~Product()
    {
        Console.WriteLine(this.name + "의 소멸자 호출");
    }
}
```

클래스 이름 앞에 틸드(~) 기호를 붙여 생성.

클래스 이름 앞에 틸드(~) 기호를 붙여 생성.

- 인스턴스의 수명이 다해 소멸될 때 호출되는 특수 메서드.
- C#은 가비지 컬렉터(Garbage Collector)가 메모리를 자동 관리하므로, 개발자가 소멸자를 직접 호출하거나 작성할 일은 매우 드뭄.

불변의 데이터: 상수 (Constants)

```
public const double PI = 3.141592;
```

✖ Error

```
Math.PI = 20;
```

CS0131: 할당식의 왼쪽은 변수, 속성 또는 인덱서여야 합니다.

- **const**: 값을 선언함과 동시에 영구적으로 고정.
- 프로그램 실행 중 절대 변경할 수 없으며, 변경 시도 시 컴파일 오류 발생.
- 특징: 인스턴스화 없이 클래스 이름으로 직접 접근(Math.PI)하며, 메서드 내부의 지역 변수로도 사용 가능.



절대적 불변성
(Absolute Immutability)

런타임 불변 보호: readonly

```
class Product
{
    private static int count;
    public readonly int id;
    public string name;
    public int price;

    public Product(string name, int price)
    {
        id = count++;
        this.name = name;
        this.price = price;
    }
}
```

선언 (Declaration)

생성자 내부 할당 (Valid)

```
Product product = new Product("이름", 100);
product.id = 10;
```

생성자 외부 변경 시도 (Error - Invalid)

상수(const)와 유사하지만, 초기화 시점에 유연성을 부여한 읽기 전용 키워드.
변수를 선언하는 시점과 생성자 내부에서만 값을 변경/할당할 수 있음. 그 이후에는 값이 영구히 잠김.

데이터 보호 매트릭스: const vs readonly

특성 (Dimension)	const	readonly
초기화 시점 (Initialization)	✓ 선언하는 즉시 무조건 할당해야 함	✓ 선언 시점 또는 생성자 에서 할당 가능
메모리 시점	✓ 컴파일 타임 (Compile-time)	✓ 런타임 (Run-time)
소속	✓ 자동으로 static 처럼 작동 (클래스 소속)	✓ 기본적으로 인스턴스 소속 (객체마다 다른 값 보유 가능)
용도	✓ 원주율, 물리 상수 등 완벽히 고정된 값	✓ 객체 생성 시 부여되는 고유 ID, 타임스탬프 등

합성: 클래스의 해부학

